

Basic Idea is select the minimum and swap it.

Eg. 13 46 24 52 20 (9) $\rightarrow$ min = 9 so swap with index 0 element

Step 1 - 9 46 24 52 20 (13)

Step 2 - 9 13 24 52 (20) 46

Step 3 - 9 13 20 52 (24) 46

Step 4 - 9 13 20 24 52 (46)

Step 5 - 9 13 20 24 46 52
— Sorted —

- ① Swap min & index 0 elem [Array from 0 to $n-1$]
- ② Swap 2nd min & index 1 elem. [Array from 1 to $n-1$]
- ③ Swap 3rd min & index 2 elem. [Array from 2 to $n-1$]
- $\vdots$
- ④ Swap last min & index $(n-2)$ elem [Array from $(n-2)$ to $(n-1)$]

Pseudocode

```

n = length of Array
for (i=0 ; i ≤ n-1 ; i++) {
    min = i
    for (j=i ; j ≤ n-1 ; j++) {  $\leftarrow$  To calculate min
        if (arr[j] < arr[min]) {
            min = j
        }
    }
    swap(i, arr[min])
}

```

class Solution:

def selectionSort(self, arr):

code here

```
for i in range(0, len(arr)):
    min = i
    for j in range(i+1, len(arr)):
        if (arr[min] > arr[j]):
            min = j
    arr[i], arr[min] = arr[min], arr[i]
```

Time Complexity:

Here the loop runs for

i=0 : n times

i=1 : n-1 times

i = 3 : n-2 times

..

..

..

Last : 2 times

Thus total = (n) + (n-1) + (n-2) + + 2 + 1

$$n(n-1)/2$$

Thus $O(n^2)$ is the time complexity